```
# ============================================================
# CSET419 – Lab 8
# Generative AI - Artistic Image Generation using GANs
# ============================================================

import torch
import torch.nn as nn
import torch.optim as optim
import torchvision
import torchvision.transforms as transforms
from torchvision.utils import save_image, make_grid
import matplotlib.pyplot as plt
import numpy as np
import os

# ----------------------------
# 1. Setup
# ----------------------------
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
print("Using device:", device)

os.makedirs("outputs", exist_ok=True)

# ----------------------------
# 2. Data Preparation
# ----------------------------
transform = transforms.Compose([
    transforms.Resize(64),
    transforms.ToTensor(),
    transforms.Normalize([0.5], [0.5])  # [-1, 1]
])

dataset = torchvision.datasets.CIFAR10(
    root="./data",
    download=True,
    transform=transform
)

loader = torch.utils.data.DataLoader(dataset, batch_size=128, shuffle=True)

# ----------------------------
# 3. Hyperparameters
# ----------------------------
latent_dim = 100
lr = 0.0002
epochs = 5

# ----------------------------
# 4. DCGAN Models
# ----------------------------
class Generator(nn.Module):
    def __init__(self):
        super().__init__()
        self.net = nn.Sequential(
            nn.ConvTranspose2d(latent_dim, 512, 4, 1, 0),
            nn.BatchNorm2d(512),
            nn.ReLU(),

            nn.ConvTranspose2d(512, 256, 4, 2, 1),
            nn.BatchNorm2d(256),
            nn.ReLU(),

            nn.ConvTranspose2d(256, 128, 4, 2, 1),
            nn.BatchNorm2d(128),
            nn.ReLU(),

            nn.ConvTranspose2d(128, 64, 4, 2, 1),
            nn.BatchNorm2d(64),
            nn.ReLU(),

            nn.ConvTranspose2d(64, 3, 4, 2, 1),
            nn.Tanh()
        )

    def forward(self, x):
        return self.net(x)


class Discriminator(nn.Module):
    def __init__(self):
        super().__init__()
```

```
        super().__init__()
        self.net = nn.Sequential(
            nn.Conv2d(3, 64, 4, 2, 1),
            nn.LeakyReLU(0.2),

            nn.Conv2d(64, 128, 4, 2, 1),
            nn.BatchNorm2d(128),
            nn.LeakyReLU(0.2),

            nn.Conv2d(128, 256, 4, 2, 1),
            nn.BatchNorm2d(256),
            nn.LeakyReLU(0.2),

            nn.Conv2d(256, 512, 4, 2, 1),
            nn.BatchNorm2d(512),
            nn.LeakyReLU(0.2),

            nn.Conv2d(512, 1, 4, 1, 0),
            nn.Sigmoid()
        )

    def forward(self, x):
        return self.net(x).view(-1)


G = Generator().to(device)
D = Discriminator().to(device)

criterion = nn.BCELoss()
opt_G = optim.Adam(G.parameters(), lr=lr, betas=(0.5, 0.999))
opt_D = optim.Adam(D.parameters(), lr=lr, betas=(0.5, 0.999))

# ----------------------------
# 5. Training DCGAN
# ----------------------------
print("Training DCGAN...")

for epoch in range(epochs):
    for i, (real, _) in enumerate(loader):
        real = real.to(device)
        batch_size = real.size(0)

        real_labels = torch.ones(batch_size).to(device)
        fake_labels = torch.zeros(batch_size).to(device)

        # ---------------------
        # Train Discriminator
        # ---------------------
        noise = torch.randn(batch_size, latent_dim, 1, 1).to(device)
        fake = G(noise)

        loss_real = criterion(D(real), real_labels)
        loss_fake = criterion(D(fake.detach()), fake_labels)

        loss_D = (loss_real + loss_fake) / 2

        opt_D.zero_grad()
        loss_D.backward()
        opt_D.step()

        # ---------------------
        # Train Generator
        # ---------------------
        output = D(fake)
        loss_G = criterion(output, real_labels)

        opt_G.zero_grad()
        loss_G.backward()
        opt_G.step()

    print(f"Epoch [{epoch+1}/{epochs}] Loss D: {loss_D:.4f}, Loss G: {loss_G:.4f}")

    # Save sample images
    with torch.no_grad():
        sample_noise = torch.randn(16, latent_dim, 1, 1).to(device)
        fake_images = G(sample_noise)
        save_image((fake_images + 1)/2, f"outputs/sample_epoch_{epoch+1}.png", nrow=4)

# ----------------------------
# 6. Generate Artistic Samples
# ----------------------------
print("Generating artistic images...")
```

```
with torch.no_grad():
    noise = torch.randn(10, latent_dim, 1, 1).to(device)
    images = G(noise)
    save_image((images + 1)/2, "outputs/random_samples.png", nrow=5)

# ----------------------------
# 7. Latent Space Interpolation
# ----------------------------
print("Generating interpolation...")

def interpolate(z1, z2, steps=10):
    result = []
    for alpha in torch.linspace(0, 1, steps):
        z = (1 - alpha) * z1 + alpha * z2
        result.append(z)
    return torch.cat(result)

z1 = torch.randn(1, latent_dim, 1, 1).to(device)
z2 = torch.randn(1, latent_dim, 1, 1).to(device)

interp = interpolate(z1, z2, 10)

with torch.no_grad():
    images = G(interp)
    save_image((images + 1)/2, "outputs/interpolation.png", nrow=10)


def show_images(path):
    img = plt.imread(path)
    plt.imshow(img)
    plt.axis("off")
    plt.show()

show_images("outputs/random_samples.png")
show_images("outputs/interpolation.png")

print("Lab Completed Successfully!")
```

```
Using device: cuda
100%|██████████| 170M/170M [00:02<00:00, 65.8MB/s]
Training DCGAN...
Epoch [1/5] Loss D: 0.2766, Loss G: 3.1072
Epoch [2/5] Loss D: 0.3941, Loss G: 2.7235
Epoch [3/5] Loss D: 0.1769, Loss G: 3.0896
Epoch [4/5] Loss D: 0.4371, Loss G: 3.1263
Epoch [5/5] Loss D: 1.2439, Loss G: 0.8721
Generating artistic images...
Generating interpolation...
```

```
Lab Completed Successfully!
```